SIMATS ENGINEERING
ASSESSMENT TOOL 2

COURSE CODE: CSA1408 **COURSE NAME: Compiler Design**
Name: Ch.B.Vinay **Register Num:**192425148
Faculty: Dr.G.Michael G

COURSE OUTCOME COVERED
CO2: Construct top-down and bottom up parsers using CFG for parsing conflicts and error recovery for efficient syntax tree generation (BL3)
Assessment Tool Weightage: 20%

Duration :60 Mins
On class
QUESTIONS: **Total Marks:50**

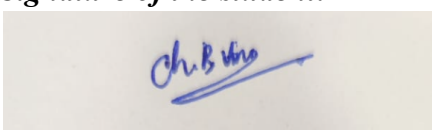
Annexure A

SHORT ANSWER TEST

Code of Conduct:

I **Ch.B.Vinay(192425148)** *(Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment.*
I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



Ch. B. Vinay
192425148.

SHORT ANSWER TEST

Q1. A lexical analyzer produces the following token stream:
id + id
Explain how the parser validates the syntactic structure of the expression.

Q2. Given the grammar:
$S \rightarrow aS \mid b$
Determine whether the string aaab belongs to the language using leftmost derivation.

Q3. Remove left recursion from the grammar:
$A \rightarrow Aa \mid b$

Q4. Find FIRST(S) for the grammar:
$S \rightarrow aA \mid b$
$A \rightarrow c \mid \varepsilon$

Q5. Determine whether the grammar is suitable for top-down parsing.
$S \rightarrow Sa \mid b$
Justify your answer.

Q6. Apply left factoring to the grammar:
$S \rightarrow$ if E then S
$S \rightarrow$ if E then S else S

Q7. Construct a parse tree for the string:
a + b
using the grammar:
$E \rightarrow E + E$
$E \rightarrow$ id

Q8. Write the recursive descent procedures for the grammar:
$S \rightarrow aA$
$A \rightarrow b$

Q9. Given the grammar:
$S \rightarrow AB$
$A \rightarrow a$
$B \rightarrow b$
Construct the predictive parsing table.

Q10. Demonstrate the first three actions of a shift-reduce parser for the input:
id + id

Q11. Identify the handle in the string:
id + id
for the grammar:
$E \rightarrow E + E$
$E \rightarrow$ id

Q12. Construct the operator precedence relations for the operators:
+
*
where * has higher precedence than +.

Q13. Explain how operator precedence parsing handles the expression:
id * id

Q14. What is an LR parser? Illustrate the parsing process using the grammar:

$S \rightarrow aA$
$A \rightarrow b$

Q15. Given the grammar:
$S \rightarrow aA$
$A \rightarrow b$
Write the steps involved in constructing an SLR parsing table.

Q16. Compute FOLLOW(A) for the grammar:
$S \rightarrow AB$
$A \rightarrow a$
$B \rightarrow b$

Q17. Construct the augmented grammar for:
$S \rightarrow a$

Q18. Find the closure of the LR(0) item:
$S' \rightarrow \cdot S$
for the grammar:
$S \rightarrow a$

Q19. Check whether the grammar is ambiguous:
$E \rightarrow E + E$
$E \rightarrow id$
using the string:
id + id + id

Q20. Construct a leftmost derivation for the string:
id + id
using the grammar:
$E \rightarrow E + E$
$E \rightarrow id$

Q21. Apply left factoring to the grammar:
$S \rightarrow aA$
$S \rightarrow aB$
$A \rightarrow b$
$B \rightarrow c$

Q22. Find FIRST(A) for the grammar:
$A \rightarrow x$
$A \rightarrow y$

Q23. Determine whether the grammar is suitable for predictive parsing.
$S \rightarrow aA \mid aB$
$A \rightarrow b$
$B \rightarrow c$
Justify your answer.

Q24. Construct recursive descent parser procedures for the grammar:
$S \rightarrow aS$
$S \rightarrow b$

Q25. Build the predictive parsing table for the grammar:
$S \rightarrow aS$
$S \rightarrow b$
Show the table entries for the terminals:
a, b, $

CO2 - AT2 - Short Answer test

...408 - Compiler Design

Ch. B. Vinay
192425148.

Dr. G. Michael.

The Parser checks id + id follows grammar rules & accepts the syntatic structure of expression

Given grammar,

S → aS/b.   o/P : aaab.

left most derivation

S → aS

S → aaS

S → aaaS

S → aaab.

3. Given Grammar:

A → Aa/b.

left recursion:

A → Aα/β.

then

A → βA'

A' → αA'/ε.

A → Aa/b.

A → bA'

A' → aA'/ε.

4. First(S):

S → aA/b

A → c/ε.

First (S) = {a, b}.

5. The grammar has left recursion So it doesn't have top-down parsing.

6.

6. Given Grammar:

$S \to$ if ϵ then S

$S \to$ if ϵ then S else S.

Left factoring:

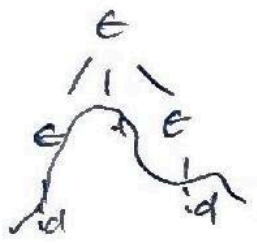
$S \to$ if ϵ then S X

$X \to$ else S / ϵ.

7. Given Grammar

$E \to E+E$

$E \to id$.



8. Given Grammar.

$S \to aA$.

$A \to b$.

```
S() {
match(a);
A();
}
A()
{
match(b);
}
```

9. Given Grammar: $S \to AB$, $A \to a$, $B \to b$

	a	b	$
S	$S \to AB$	-	-
A	$a \to A$	-	-
B	-	$B \to b$	-

10. Given, input : id+id.

shift id

Reduce $id \to E$.

shift +.

11. Given string id+id.

Grammar : $E \to E+E$

$E \to id$.

handle in string is id

12. Given precedence.

	+	*
+	$\gtrdot$	$\lessdot$
*	$\gtrdot$	$\gtrdot$

13. Given Expression: id*id.

i. input : $id*id$$

shift id
Reduce $id \to E$
shift * shift id
Reduce $id \to E$
Reduce $E*E \to E$ Accept

14. LR Parser is an efficient bottom up syntax analyzer that reads input from left to right to Produce right most Derivation.

Shift a

shift b.

Reduce A→b.

Reduce S→aA.

Accept.

15. Steps involved in constructing an SLR Parsing table:

→ step 1: Construct the Augumented Grammar.

→ build DFA of item sets.

→ Compute first & follow sets.

→ Fill Action table & GOTO Table

→ Accept if no Conflict is exists.

16. Given Grammar,

S→AB. , A→a, B→b.

Follow(A) - First(B) = b.

∴ Follow(A) = b.

17. Given Grammar,

S→a

Augumented Grammar:

S→S'

S→.a

18. Given Grammar:

LR(1) S'→ .S

S→a

LR(0):

S'→ .S

S→ .a.

19. Given String id+id+id

Grammar : E→E+E

E→id.

Here the Grammar ~~string~~ Produces more than one Parse tree so it is ambigious grammar

20. Given

Grammar: E→E+E, E→id

left most Derivation:

E→E+E

E→id+E

E→id+id.

21. Given Grammar,

S→aA. A→b

S→aB B→c

left factoring:

S→aS'

S'→A/B.

A→b

B→c

22. Given Grammar,

$A \to x \quad A \to y$.

First(A) = {x, y}.

23. Given Grammar:

$S \to aA \mid aB$.

$A \to b$

$B \to c$

Not suitable for predictive parsing because both productions begin with a causing FIRST/FIRST conflict. so for this left recursion is required

24. Given.

$S \to aS$

$S \to b$.

```
S() {
if ( input == 'a') {
match(a);

}
else {
 match(b);
 }
}
```

25.

$S \to aS$

$S \to b$.

	a	b	$
S	$S \to aS$	$S \to b$.	-